Certified Federated Unlearning for IoT: Ledger-Anchored Revocation with Anytime-Valid Forgetting Certificates

Quang-Vinh Dang

***Abstract*—Federated learning lets IoT devices train a shared model without exporting data, but a device's influence stays in the model after it leaves, and regulation, decommissioning and poisoned contributions all demand removal. Retraining for every request is too costly for constrained fleets, and existing federated unlearning checks are empirical, intrusive or limited to logging the request. We present a federated unlearning and governance layer for the FORTRESS-FL robust aggregation system. A hash-chained ledger with Merkle roots, per-round checkpoints of model and reputation state, and a consent registry make revocation traceable. Clients plant paired ghost canaries before training; after a revocation wave an anytime-valid e-process verifier (VOUCH) issues an (ϵ, α) forgetting certificate without retrained references or backdoor markers. A verify-then-escalate policy tries cheap unlearning operators first and releases the first certified, utility-preserving model, with a proof that the error budget α holds over the whole ladder. On 60 simulated clients with partial participation, an un-unlearned model was flagged in 40 of 40 calibration runs and exactly retrained models were never flagged; the policy certified every wave at 31% lower mean client energy than retraining, but with a worst case 44% above it. Waves of 20 clients cost 29 times less energy per client than serving requests one at a time, and ledger overhead was about 1% of update traffic.**

***Index Terms*—Federated unlearning, model governance, provenance, anytime-valid inference, Byzantine robustness, Internet of Things.**

I. Introduction

FEDERATED learning lets IoT devices train a shared model without exporting raw data [1], but a device's data keeps influencing the global model long after the device has left. Data protection law gives owners a right to erasure [2], devices are decommissioned or change owner, and poisoned contributions are discovered after the fact. Retraining from scratch for every request costs the whole fleet its original training energy and radio traffic, which is not realistic at IoT scale.

Federated unlearning removes a client's influence without full retraining [3]–[6]. Two problems remain for deployment. First, the operator has to *show* that the influence is gone. Existing checks mark the leaving client's data with backdoor-style markers [7], rely on fingerprints [8], or record requests on a blockchain [9], [10]; the first two are empirical and the third records that unlearning was requested rather than whether it worked. Second, verification is almost never tied to cost: an operator who must pay for a certificate has little reason to choose the cheap method over the expensive one unless the certificate tells them whether the cheap method was enough.

Q.-V. Dang is with RMIT University Vietnam. E-mail: vinh.dang3@rmit.edu.vn.

We address both problems in one system built on the FORTRESS-FL robust aggregation pipeline [11] and on the paired-canary verifier VOUCH [12]. Our contributions are:

1) **Ledger-anchored revocation.** A hash-chained provenance ledger with per-round Merkle roots over client updates, a consent registry, and per-round checkpoints of the model *and* of the reputation state, since reputations also remember past contributors. A rollback record names the clean checkpoint, so an auditor can check lineage and bind a certificate to the ledger head and the released model hash.
2) **Federated paired-canary certificates.** Clients plant paired ghost canaries before training and commit to them in the ledger. After a revocation wave, an anytime-valid e-process verifier [12] issues or withholds an (ϵ, α) forgetting certificate. The null is generated by the clients' own coins, so no retrained reference model, shadow models or backdoor markers are needed. We adapt the protocol to client-level revocation, partial participation and batched waves.
3) **Verify-then-escalate.** A cost ladder from update subtraction to rollback in which the certificate is the acceptance test. We prove that splitting the error budget over the rungs keeps the probability of ever certifying a leaky model below α (Theorem 1), and we add a utility guard because forgetting that destroys the model is not a result.
4) **Evaluation on resource-constrained fleets.** We quantify client energy and traffic under an explicit cost model, and study non-IID data, partial participation, DP noise, model capacity, poisoned and Byzantine clients, wave size, and ledger overhead and tamper detection.

What is new relative to FORTRESS-FL and VOUCH

This paper builds on two earlier works by the same author, and we state precisely what it adds. *FORTRESS-FL* [11] is a training-time system: Pedersen-committed updates, spectral Byzantine detection, reputation-weighted aggregation and Gaussian noise. It has no notion of removing a client after training, keeps no per-round state, and offers no audit. *VOUCH* [12] is a verifier for centralised (language-model) unlearning: it assumes a single trained model and a forget set, and certifies it with paired canaries and e-processes. It has no federated protocol, no client revocation, no unlearning operators,

TABLE I
CONTRIBUTION OF EACH WORK.

	FORTRESS-FL [11]	VOUCH [12]	This paper
Setting	federated training	centralised LMs	federated, IoT
Byzantine robust	yes	n/a	inherited, tested during unlearning
Client removal	no	forget-set only	client waves, operators
Per-round state	no	no	model, reputation, updates
Provenance	commitments	manifest hash	ledger, Merkle, consent
Verification	none	e-process certificate	certificate per wave, ledger-bound
Cost-aware choice	no	no	verify-then-escalate
Energy/traffic	no	GPU passes	client model

and no cost model. Table I lists what each contributes and what is added here. The new elements are: (i) checkpointing of model *and* reputation state, the ledger, the consent registry and the unlearning operators on top of FORTRESS-FL; (ii) a federated canary protocol in which clients hold and commit their own pairs and certificates are issued per revocation wave under partial participation; (iii) the verify-then-escalate policy with its error accounting (Theorem 1), which neither earlier work has; (iv) the energy and traffic model; and (v) experiments on non-IID, partial participation, DP noise, capacity, poisoned and Byzantine clients. The e-process verifier and its coverage theorem are used unchanged from VOUCH and are not claimed as new.

II. RELATED WORK

Federated unlearning. FedEraser rebuilds the model from stored historical updates with calibration [3]; other methods use knowledge distillation [5], gradient-based removal [4], noise-based certified removal [13] or on-device teachers [14]. FedRecover estimates client updates to recover from poisoning [15]. Exact approaches isolate data in shards [16]. Surveys [6] note that cost, verification and non-IID behaviour are open. Our operators (subtraction of stored updates, rollback to the last clean checkpoint, replay, sharding) are standard; the contribution is the certificate that decides between them.

Verifying unlearning. Backdoor-marker verification in federated settings [7] detects removal through the behaviour of the marker, which requires modifying the data and gives no error guarantee. PrivEraserVerify reports fingerprint-based verification with indistinguishability claims [8]. Membership inference is the usual statistical probe [17], [18], but a threshold on an attack score is not a calibrated test, and unlearning definitions need to be auditable [19]. Certified removal gives guarantees for particular algorithms and convex models [20]; ours is a property of the released model, whichever algorithm produced it.

Provenance and ledgers in FL. Blockchain-based designs record updates and unlearning events for traceability [9], [10]. We use a lightweight hash chain with Merkle inclusion proofs [21] and, unlike a log of requests, couple it to a statistical test of the result.

Anytime-valid testing. E-processes and betting martingales give tests valid under optional stopping [22], [23]. VOUCH applies them to centralised unlearning with paired canaries [12]. We reuse its verifier and theorems as a component and cite them rather than restate them; the federated protocol, ladder, ledger and evaluation are new here.

III. PROBLEM AND THREAT MODEL

N clients, each with a local data set $\mathcal{D}_c$, train a model $\theta \in \mathbb{R}^d$ with T rounds of robust aggregation. In round t a subset of clients participates, each independently with probability p. A *revocation wave* $\mathcal{W}$ is a set of clients whose consent is withdrawn (or whose updates are found poisoned). The operator must release a model θ_u that no longer carries their influence, and produce evidence that a third party can check.

The operator is honest-but-verifiable: it executes the protocol but may take shortcuts to save cost, which is exactly the temptation the audit has to catch. A minority of clients may be Byzantine during training. Clients are not trusted to self-report forgetting. What is certified is the *residual advantage* that a declared class of scores can extract from the canary cohort of the wave on the released model; it is not erasure in parameter space, which no behavioural test can establish.

IV. SYSTEM

A. Training and provenance

Training follows FORTRESS-FL: each client commits to its clipped update with a Pedersen commitment [24], the server verifies openings, clusters updates spectrally to flag a Byzantine group, aggregates the rest with reputation weights, adds Gaussian noise and updates reputations [25], [26]. We gate the cluster split: a group is flagged only if its cross-similarity to the rest is lower than the within-group similarity by more than a margin, so IID honest rounds are not split arbitrarily.

After each round the ledger appends a record containing the round index, the participant list, the Merkle root over leaves $H(\text{client} \,\|\, H(\text{update}))$, the aggregation weights, the flagged clients and the hash of the new model. Each record includes the hash of its predecessor. The server also stores the model and the reputation vector of every round (the checkpoint), and, optionally, the clipped updates themselves. A *consent registry* tracks revocations and marks them fulfilled only after a rollback record exists.

B. Unlearning operators and cost model

For a wave $\mathcal{W}$ let k_0 be the first round in which any member of $\mathcal{W}$ participated according to the ledger, and θ_{k_0-1} its checkpoint. The operators are: *retrain* (restart from θ_0 on the retained clients); *rollback* (restore θ_{k_0-1} and the reputations of that round, then re-run rounds $k_0, \dots, T$ without $\mathcal{W}$); *subtract* ($\theta_T + \eta \sum_t \sum_{c \in \mathcal{W}} w_{c,t} \tilde{g}_{c,t}$ using the stored clipped updates, and zero reputation); *subtract+R* (subtract, then R fresh rounds); *replay* (re-apply stored updates of the retained clients from the

checkpoint, no client computation); and *sharded* (clients split into cohorts, one cohort retrained). After a rollback the ledger forks off the contaminated checkpoints and stored updates are purged so that the revoked clients' influence does not survive in the server's own records.

Proposition 1. *If clients participate independently with probability p in each round, the first round k_0 in which any of m revoked clients participates satisfies $\Pr[k_0 > r] = (1-p)^{rm}$, so $\mathbb{E}[k_0] = 1/(1-(1-p)^m)$ (about 1 for $m \geq 5$ at $p = 0.6$) and rollback saves at most about one round out of T.*

This explains why rollback matched retraining in our runs and why savings must come from approximate operators that are then verified. Batching helps differently: rollback cost does not grow with $|\mathcal{W}|$, so the cost per revoked client falls as $1/|\mathcal{W}|$.

Cost model. A client round costs $4n_c d$ FLOPs, uploads $4d + 64$ bytes (float32 update and a 512-bit commitment) and downloads $4d$ bytes. Energy is $e_{\text{flop}} \cdot \text{FLOPs} + e_{\text{bit}} \cdot 8 \cdot \text{bytes}$ with $e_{\text{flop}} = 1$ nJ and $e_{\text{bit}} = 100$ nJ. These constants are assumptions typical of low-power radios and processors, not measurements; the ratio between methods follows the round count and is insensitive to them. Canary reveal adds $2m(d{+}1) \cdot 4$ bytes per revoked client once.

C. Paired canaries and the certificate

Before training, each client draws m twin pairs of exchangeable records (Gaussian features, fair random labels), flips a coin per pair to pick the in-twin, adds only the in-twin to its data, keeps the ghost, and publishes a digest of the manifest in the ledger. After a wave, the verifier receives the pairs of $\mathcal{W}$ and scores the released model on both twins with a declared class of scores (negative loss and signed margin). If θ_u carries no influence of the in-twins, each sign of the score difference is a fair coin; betting e-processes on these signs yield (i) a confidence sequence for the residual advantage Δ, (ii) an (ϵ, α) certificate once evidence that $|\Delta| < \epsilon$ is conclusive, and (iii) a revocation alarm when the advantage is positive [12]. In a deployment the reveal order of the pairs is drawn from a public beacon; our experiments use a fixed seed so that runs are reproducible.

D. Verify-then-escalate

Let rungs $1, \ldots, K$ be operators ordered by cost. The policy runs rung k on a scratch copy, tests its output at level α/K, and releases the first rung that is certified and keeps validation accuracy within δ of the pre-unlearning model. If the last rung does not certify, it is released flagged UNCERTIFIED.

Theorem 1. *Let L be the set of rungs whose output has realised cohort advantage at least ϵ in some score of the declared class. The probability that the policy ever releases a rung in L as certified is at most α, for any (data-dependent) stopping rule inside each rung's test and any dependence between rungs.*

Proof. By the coverage theorem of the verifier [12], each rung $k \in L$ is certified with probability at most α/K. A union bound over at most K rungs gives α. The utility guard only rejects further rungs and cannot increase the probability of releasing a rung in L. □

V. Experiments

A. Setup

Federation. $N = 60$ clients hold linear classifiers on $d = 1024$-dimensional features, the regime of a linear head on frozen on-device embeddings. Data are ten Gaussian latent classes with binary labels (class parity) and 8% label noise, split by a Dirichlet(α=1) partition; each client has 12 real samples and plants $m = 6$ canary pairs. Each round every client participates with $p = 0.6$; $T = 60$ rounds, learning rate 1, update clipping at norm 1, no DP noise unless stated. A wave revokes $W = 20$ clients (120 pairs); $\epsilon = 0.2$, $\alpha = 0.05$. Test accuracy of the trained model is 0.98. Main results use 10 seeds; sweeps, wave-size and poisoning experiments use 5. All numbers are regenerated by `experiments/unlearning_experiments.py`. The audit's reveal order is seeded for reproducibility (a deployment would use a beacon). The synthetic benchmark is the only data set; we discuss this in Section VI.

Cost. Energy and traffic come from the model in Section IV-B. With the assumed constants radio traffic dominates energy, so energy tracks bytes.

B. Calibration of the audit

We first check the verifier on the two cases where the truth is known (40 seeds, N=40, T=30, W=20). The model before unlearning was flagged REVOKED in 40 of 40 runs (upper bound on the advantage 0.78), and the exactly retrained model was ISSUED in 40 of 40 runs with no false revocation. A leaky model was therefore never certified and an exactly unlearned one was never accused.

C. Unlearning operators under the certificate

Table II and Fig. 1 compare the operators on the same wave. The un-unlearned model is flagged in 10 of 10 runs. Rollback restored round 0 in every run, because with 20 revoked clients and $p = 0.6$ some member participates in round 1 (Proposition 1), so rollback and retraining are the same here. Plain subtraction and replay cost no client energy but cost accuracy (0.62), and the certificate cannot see that: it was issued in 4 of 10 runs, which is why the policy needs the utility guard. Subtraction followed by R recovery rounds restores accuracy (0.98); more recovery rounds raise the share of certified runs (4, 4 and 6 of 10 for $R = 5, 10, 20$, and 8 of 10 for retraining) at 0.8, 1.6, 3.2 and 9.6 J. No operator was certified in all runs: with 120 pairs the test is near its resolution at $\epsilon = 0.2$, and even retraining ended UNDETERMINED twice. UNDETERMINED means the evidence was not conclusive, not that the model leaks.

D. Verify-then-escalate

The ladder was subtract, subtract+5, subtract+20, rollback, each tested at $\alpha/4$, with a validation set of 300 samples and a 3-point accuracy guard. It certified 10 of 10 waves at accuracy

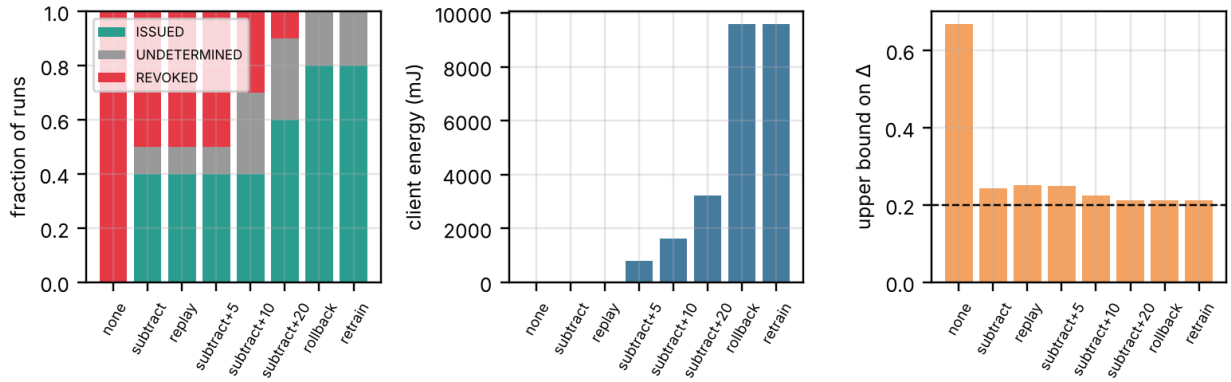


Fig. 1. Certificate outcomes, client energy and upper confidence bound on the residual advantage per operator (10 seeds). The dashed line is $\epsilon = 0.2$.

TABLE II
OPERATORS ON ONE WAVE OF 20 CLIENTS (10 SEEDS). I/U/R: RUNS WITH CERTIFICATE ISSUED, UNDETERMINED, REVOKED. ENERGY IS CLIENT-SIDE.

Operator	I/U/R	Accuracy	Rounds	Energy (J)
none	0/0/10	0.98	0	0
subtract	4/1/5	0.62	0	0
replay	4/1/5	0.62	0	0
subtract+5	4/1/5	0.98	5	0.81
subtract+10	4/3/3	0.98	10	1.64
subtract+20	6/3/1	0.98	20	3.24
rollback	8/2/0	0.98	60	9.58
retrain	8/2/0	0.98	60	9.58

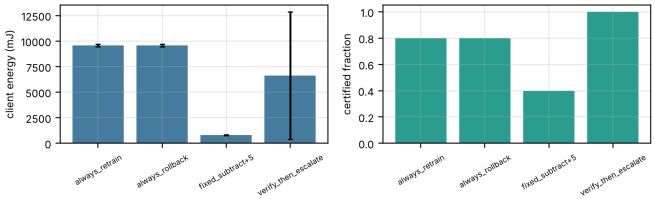


Fig. 2. Policies on the same waves: mean client energy (error bars: s.d.) and fraction of waves certified.

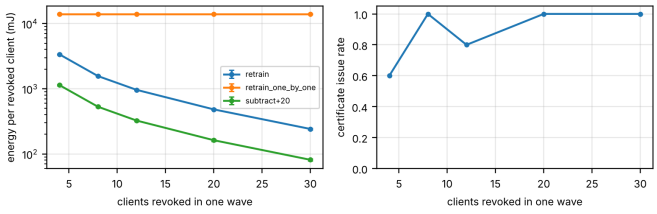


Fig. 3. Energy per revoked client versus wave size (left) and issue rate for the retrained model (right, 5 seeds).

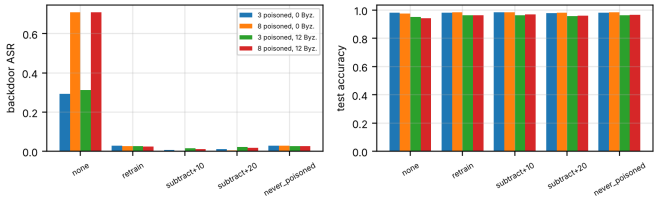


Fig. 4. Backdoor success rate and test accuracy before and after unlearning, with 3 or 8 poisoned clients and 0 or 12 Byzantine clients (5 seeds).

0.98, against 8 of 10 for always retraining at the full level. It released subtract+5 four times, subtract+20 twice and rollback four times. Mean client energy was 6.6 J against 9.6 J for retraining (31% less, 41 instead of 60 rounds on average), but the cost is bimodal: from 0.8 J when the first rung succeeded to 13.8 J, 44% above retraining, when all four rungs were tried because failed attempts are paid for. Fixing subtract+5 as the only operator was cheapest (0.8 J) and was certified in 4 of 10 runs only (Fig. 2).

E. Wave size and batching

Retraining cost per revoked client falls from 3.4 J for waves of 4 to 0.24 J for waves of 30, because rollback cost does not grow with the wave (Fig. 3). Serving 20 requests one at a time costs 14.0 J each (measured on three requests and extrapolated), 29 times more than a single batched wave. The price is delay and the need for enough pairs: the retrained model was certified in 3 of 5 runs at $W{=}4$ (24 pairs) and in 5 of 5 at $W \in \{8, 20, 30\}$, with 4 of 5 at $W{=}12$. Revealing the canaries sends $2m(d{+}1) \cdot 4$ bytes per client, 0.98 MB for $W{=}20$. In practice requests have to be held until a wave is large enough, which is compatible with the response periods of data protection law but should be agreed with the regulator.

F. Poisoned and Byzantine clients

Three or eight clients in the wave stamped a trigger on half of their data (relabelled to the target class); in half of the runs 12 further clients were Byzantine sign-flippers. Before unlearning the backdoor attack success rate was 0.29 with three poisoned clients and 0.71 with eight (0.31 and 0.71 with Byzantine clients present). After subtraction with 10 recovery rounds it fell to 0.007 and 0.002 (0.016 and 0.010 with Byzantine clients), against 0.027 for retraining and 0.029 for a model never trained on them, at 2.1–2.3 J versus 12.5–13.8 J, with accuracy unchanged (Fig. 4). The rate of the clean model is the false-trigger floor. Recovery was as effective with the Byzantine clients present.

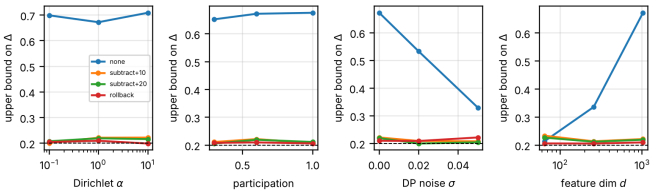


Fig. 5. Upper bound on the residual advantage versus Dirichlet α, participation, DP noise and feature dimension (5 seeds). Dashed: $\epsilon = 0.2$.

G. Heterogeneity, participation, noise and capacity

Fig. 5 shows the bound for the un-unlearned model and for three operators over four factors (5 seeds). The approximate operators were certified more often with stronger heterogeneity (subtract+10: 5, 2 and 3 of 5 runs at $\alpha = 0.1, 1, 10$), lower participation (4, 2 and 3 of 5 at $p = 0.3, 0.6, 1$) and more DP noise (2 and 5 of 5 at $\sigma = 0$ and 0.02). We did not isolate the cause; fewer exposures of a canary and noise on the update both plausibly reduce what the model retains. Accuracy fell with noise (0.98 to 0.94 at $\sigma = 0.02$) for every operator. Capacity matters for audit power: at $d = 64$ the un-unlearned model was certified in 3 of 5 runs, because the model cannot memorise the canaries, so the advantage really is below ϵ there; at $d \geq 256$ it was flagged in all runs.

H. Provenance overhead and tamper detection

For 60 rounds the ledger holds 61 records and 101 kB (1.7 kB per round), about 1.1% of update upload traffic. Verifying the whole chain took 1.6 ms and a Merkle inclusion proof (5–6 hashes) about 5 μs. In 3 seeds $\times$ 200 random edits of a record field (root, model hash, round, weights, flagged list), the chain check detected all 600; an HMAC-signed certificate failed on any change of its status; a forged rollback record that kept training the revoked wave was rejected by the lineage audit; and the lineage audit passed for every operator in the main experiment. The server's checkpoints took 417 kB, but stored updates 15.2 MB, which is the main storage cost of the cheap operators.

VI. Discussion and Limitations

What a certificate means. It bounds the advantage that two declared scores extract from the canary cohort of the wave on the released model, and holds at any stopping time. It is not erasure in parameter space, and it does not cover attacks outside the score class. Canaries must be planted before training, so models trained without them cannot be certified retroactively, and a provider that treats canary records specially defeats the audit. The ledger audit certifies lineage only; the statistical certificate and the lineage check answer different questions and both are reported.

Limits of the evidence. All results use one synthetic benchmark with linear models on high-dimensional features, 5–10 seeds, simulated clients and a modelled energy cost. We have not run neural networks, real IoT data or measured device energy, and the audit's power depends on the model's capacity to memorise. The certificate is per wave, so one client cannot be certified alone at this canary budget, and the procedure for small waves needs a larger canary budget per client. The escalation policy saves energy on average but can cost more than retraining in the worst case, so a deployment should cap the number of attempts. Sample- and class-level requests are implemented for rollback and retraining but neither tested nor evaluated here, and feature-level forgetting is not supported.

Regulatory alignment. The ledger and the certificate give a data controller a verifiable record that a revocation was honoured; whether this satisfies a specific legal duty is a question for counsel, and we make no such claim.

VII. Conclusion

We combined a hash-chained provenance ledger, per-round state checkpoints, paired-canary e-process certificates and a cost ladder into a federated unlearning workflow whose acceptance test is statistical rather than empirical. On a simulated fleet the audit never accused an exactly retrained model and never accepted a leaky one, the ladder certified every wave at 31% lower mean energy than retraining, and batching revocations cut per-client cost by an order of magnitude. Next steps are neural models, real IoT data, measured energy and per-client certificates with a smaller canary budget.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017.

[2] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation), art. 17," 2016.

[3] G. Liu, X. Ma, Y. Yang, C. Wang, and J. Liu, "FedEraser: Enabling efficient client-level data removal from federated learning models," in *Proc. IEEE/ACM Int. Symp. Quality of Service (IWQoS)*, 2021.

[4] A. Halimi, S. Kadhe, A. Rawat, and N. Baracaldo, "Federated unlearning: How to efficiently erase a client in FL?" *arXiv preprint arXiv:2207.05521*, 2022.

[5] C. Wu, S. Zhu, and P. Mitra, "Federated unlearning with knowledge distillation," *arXiv preprint arXiv:2201.09441*, 2022.

[6] Z. Liu, Y. Jiang, J. Shen, M. Peng, K.-Y. Lam, X. Yuan, and X. Liu, "A survey on federated unlearning: Challenges, methods, and future directions," *ACM Computing Surveys*, 2024.

[7] X. Gao, X. Ma, J. Wang, Y. Sun, B. Li, S. Ji, P. Cheng, and J. Chen, "VeriFi: Towards verifiable federated unlearning," *IEEE Trans. Dependable and Secure Computing*, 2024.

[8] P. Goswami, M. K. Islam, and A. Yeafi, "PrivEraserVerify: Efficient, private, and verifiable federated unlearning," *arXiv preprint arXiv:2604.12348*, 2026.

[9] X. Zuo, M. Wang, T. Zhu, L. Zhang, S. Yu, and W. Zhou, "Federated learning with blockchain-enhanced machine unlearning: A trustworthy approach," *arXiv preprint arXiv:2405.20776*, 2024.

[10] "BlockFUL: Enabling unlearning in blockchained federated learning," *arXiv preprint arXiv:2402.16294*, 2024.

[11] Q.-V. Dang, "FORTRESS-FL: Federated operator resilient trustworthy resource efficient secure slice learning," Software and manuscript, https://github.com/vinhqdang/fortress-fl, 2025.

[12] ——, "VOUCH: Anytime-valid certificates for machine unlearning via paired-canary hypothesis-betting," *Machine Learning*, 2026, accepted.

[13] "Privacy-preserving federated unlearning with certified client removal," *arXiv preprint arXiv:2404.09724*, 2024.

[14] "FedQUIT: On-device federated unlearning via a quasi-competent virtual teacher," *arXiv preprint arXiv:2408.07587*, 2024.

[15] X. Cao, J. Jia, Z. Zhang, and N. Z. Gong, "FedRecover: Recovering from poisoning attacks in federated learning using historical information," in *Proc. IEEE Symp. Security and Privacy*, 2023.

[16] L. Bourtoule, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *Proc. IEEE Symp. Security and Privacy*, 2021.

[17] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *Proc. IEEE Symp. Security and Privacy*, 2017.
[18] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *Proc. IEEE Computer Security Foundations Symp.*, 2018.
[19] A. Thudi, H. Jia, I. Shumailov, and N. Papernot, "On the necessity of auditable algorithmic definitions for machine unlearning," in *Proc. USENIX Security Symp.*, 2022.
[20] C. Guo, T. Goldstein, A. Hannun, and L. van der Maaten, "Certified data removal from machine learning models," in *Proc. ICML*, 2020.
[21] R. C. Merkle, "A digital signature based on a conventional encryption function," in *Proc. CRYPTO*, 1987.
[22] A. Ramdas, P. Grünwald, V. Vovk, and G. Shafer, "Game-theoretic statistics and safe anytime-valid inference," *Statistical Science*, vol. 38, no. 4, 2023.
[23] A. Ramdas and R. Wang, *Hypothesis Testing with E-values*. Foundations and Trends in Statistics (to appear), 2024.
[24] T. P. Pedersen, "Non-interactive and information-theoretic secure verifiable secret sharing," in *Proc. CRYPTO*, 1991.
[25] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. NeurIPS*, 2017.
[26] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*. Now Publishers, 2014.